

LAB 6

PIPES AND THREADS

Lab Overview:

For this lab, we will learn to use **threads**, **mutexes**, **pipes**, and **Valgrind** in C.

A **thread** is a smaller unit of execution inside a process. Multiple threads can run at the same time and share the same memory space. Since threads share memory, problems can arise when multiple threads access or modify the same variable simultaneously. This problem is called a **race condition**.

To prevent race conditions, this lab uses a **mutex lock**. A mutex makes sure that only one thread can access a shared variable at a time.

This lab also introduces **pipes**. A pipe is a communication mechanism that allows one part of a program to write data and another part to read data. In this lab, worker threads will write messages to a pipe, and the main thread will read and print those messages.

You will complete two programs:

1. *threads.c*
2. *pipes.c*

You will also use **Valgrind** to check your programs for memory/resource issues and possible threading problems.

Core tasks:

1. Complete *threads.c*.
2. Complete *pipes.c*.
3. Compile both programs using the Makefile.
4. Run both programs and verify the output.
5. Run both programs with Valgrind.
6. Submit source files and screenshots.

Task details:

Task -1:threads.c

In `threads.c`, you will create multiple threads that update a shared counter.

Because all threads access the same counter, you must protect the counter using a mutex.

Requirements for `threads.c`

Your program must:

1. Create at least **10 threads**.
2. Use `pthread_create()` to create each thread.
3. Use `pthread_join()` to wait for all threads to finish.
4. Use a shared global counter.
5. Use `pthread_mutex_lock()` before updating the counter.
6. Use `pthread_mutex_unlock()` after updating the counter.
7. Print clear messages showing when each thread starts and finishes.
8. Print the final counter value.

Expected output:

```
Thread 0 started.  
Thread 1 started.  
Thread 2 started.  
Thread 3 started.  
Thread 4 started.  
Thread 5 started.  
Thread 6 started.  
Thread 7 started.  
Thread 8 started.  
Thread 9 started.
```

```
Thread 0 is updating the shared counter.  
Thread 0 finished.  
Thread 1 is updating the shared counter.  
Thread 1 finished.  
...
```

```
Final counter value: 1000
```

Task -2: pipes.c

In `pipes.c`, you will use threads and a pipe.

Each worker thread will perform work, update the shared counter, and then write a message to the pipe. The main thread will read messages from the pipe and print them.

Requirements for `pipes.c`

Your program must:

1. Create a pipe using `pipe()`.
2. Create at least **10 threads**.
3. Use `pthread_create()` to create each thread.
4. Use `pthread_join()` to wait for each thread.
5. Use a mutex to protect the shared counter.
6. Have each thread write a message to the pipe.
7. Have the main thread read messages from the pipe.
8. Close pipe file descriptors properly.
9. Print the final counter value.

Expected output:

```
Thread 0 started.  
Thread 1 started.  
Thread 2 started.  
...
```

```
Thread 0 updated the shared counter.  
Thread 0 wrote a message to the pipe.  
Thread 0 finished.
```

```
Message from pipe: Thread 0 completed work.  
Message from pipe: Thread 1 completed work.  
Message from pipe: Thread 2 completed work.  
...
```

```
Final counter value: 1000
```

Valgrind:

make clean && make

valgrind --leak-check=full --show-leak-kinds=all ./threads

valgrind --leak-check=full --show-leak-kinds=all ./pipes

Submission requirements:

Here is what you have to submit:

1. Threads.c
2. Pipes.c
3. Screenshot of both files compiled with the expected output.
4. Valgrind output screenshot

Naming convention for the zip file:

UoID_duckID_LabX (an example is given below)

- First Name : Dewi
- Last Name: Yokelson
- Submission for: Lab6
- So the name of the zip/tar file should be:

95xxxx_Dewi_Yokelson_Lab6.zip